

Санкт-Петербургский политехнический университет Петра Великого
Институт прикладной математики и механики
Высшая школа прикладной математики и вычислительной физики

Параллельные вычисления

Лабораторная работа №2

«Решение задачи Дирихле для уравнения Пуассона»

Работу выполнила:
студентка II курса
магистратуры
Добрецова Е.В.
Группа:
5040102/40101
Преподаватель:
Козлов К.Н.

Санкт-Петербург
2026

Содержание

1. Постановка задачи	3
2. Алгоритм метода и условия его применимости	3
2.1. Метод Зейделя для задачи Дирихле для уравнения Пуассона	3
2.2. Одномерная столбцовая декомпозиция	4
2.3. Технология MPI	4
3. Предварительный анализ задачи и проверка условий применимости метода	5
4. Тестовый пример с детальными расчётами	5
5. Подготовка контрольных тестов для иллюстрации метода	8
6. Модульная структура программы	9
7. Численный анализ решения задачи	10
7.1. Графики решения задачи	10
7.2. Исследование параллельной реализации	11
8. Выводы	14
9. Примечания	15

1. Постановка задачи

Требуется решить задачу Дирихле для уравнения Пуассона:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = (2 - y^2)\cos x \quad (1)$$

в области G с границей Γ , заданной следующим образом:

$$\overline{G} = \{(x, y) \mid 0 \leq x \leq \pi, 0 \leq y \leq 1\}; \quad (2)$$

На границе прямоугольника G известно значение решения (т.е. задано граничное условие первого рода):

$$\begin{cases} u(0, y) = y^2, 0 \leq y \leq 1; u(x, 0) = 0, 0 \leq x \leq \pi \\ u(\pi, y) = -y^2, 0 \leq y \leq 1; u(x, 1) = \cos x, 0 \leq x \leq \pi \end{cases} \quad (3)$$

Отметим, что задача (1)-(3) имеет точное решение

$$u_{exact} = y^2 \cos(x).$$

При решении задачи требуется использовать метод Зейделя с одномерной столбцовой декомпозицией для параллелизации вычислений.

Также необходимо провести анализ данной параллельной реализации, выполнив исследования следующих зависимостей:

1. зависимость времени выполнения от размера изображения;
2. зависимость времени выполнения от числа параллельных процессов/потоков;
3. зависимость ускорения от числа параллельных процессов/потоков;
4. зависимость эффективности параллелизации от числа параллельных процессов/потоков.

2. Алгоритм метода и условия его применимости

2.1. Метод Зейделя для задачи Дирихле для уравнения Пуассона

Для численного решения задачи (1) используется равномерная разностная сетка с шагами h_x и h_y по координатным направлениям. В узлах сетки (x_i, y_i) вводится сеточная функция

$$v_{i,j} \approx u(x_i, y_j), \quad i = \overline{1, N_x}, j = \overline{1, N_y},$$

при этом значения решения в граничных узлах считаются известными.

Аппроксимация оператора Лапласа центральными разностями первого порядка приводят к пятиточечному разностному шаблону:

$$\frac{v_{i+1,j} - 2v_{i,j} + v_{i-1,j}}{h_x^2} + \frac{v_{i,j+1} - 2v_{i,j} + v_{i,j-1}}{h_y^2} = f_{i,j} \quad (4)$$

Выразим из (4) $v_{i,j}$:

$$v_{i,j} = \frac{\frac{v_{i+1,j} + v_{i-1,j}}{h_x^2} + \frac{v_{i,j-1} + v_{i,j+1}}{h_y^2} - f_{i,j}}{\frac{2}{h_x^2} + \frac{2}{h_y^2}}.$$

Итерационная формула Зейделя для разностной аппроксимации уравнения Пуассона имеет вид:

$$v_{i,j}^{(k+1)} = \frac{\frac{v_{i+1,j}^{(k)} + v_{i-1,j}^{(k+1)}}{h_x^2} + \frac{v_{i,j-1}^{(k)} + v_{i,j+1}^{(k+1)}}{h_y^2} - f_{i,j}}{\frac{2}{h_x^2} + \frac{2}{h_y^2}}, \quad (5)$$

где k - номер итерации, а h_x и h_y - шаги сетки по координатным направлениям.

Итерационный процесс продолжается до выполнения условия сходимости:

$$\max_{i,j} |v_{i,j}^{(k+1)} - v_{i,j}^{(k)}| < \varepsilon,$$

где ε - заданная точность.

2.2. Одномерная столбцовая декомпозиция

На Рис. 2.1 показана иллюстрация одномерной столбцовой декомпозиции расчётной области, используемой для параллельной реализации метода Зейделя. Исходная область разбивается на вертикальные подобласти, каждая из которых обрабатывается отдельным параллельным процессом. Заштрихованные области соответствуют фиктивным ячейкам, предназначенным для хранения граничных значений сеточной функции, получаемых от соседних процессов.

Декомпозиция области позволяет выполнять пересчёт значений сеточной функции во внутренних узлах каждой подобласти параллельно. Для обеспечения корректности вычислений на каждой итерации осуществляется обмен граничными значениями между соседними подобластями с использованием средств MPI.

2.3. Технология MPI

Для параллельной реализации алгоритма в данной работе используется стандарт MPI (Message Passing Interface), предназначенный для программирования распределённых вычислений. В рамках работы MPI используется для распределения подобластей между процессами, обмена граничными значениями сеточной функции и синхронизации итераций метода Зейделя.

3. Предварительный анализ задачи и проверка условий применимости метода

Исследуемое дифференциальное уравнение в частных производных (1) относится к линейным дифференциальным уравнениям эллиптического типа (уравнение Пуассона). Задача рассматривается в замкнутой прямоугольной области (2).

Функция правой части $f(x, y) = (2 - y^2) \cos(x)$ является непрерывной и бесконечно дифференцируемой в области $\bar{G}$ ($f \in C^\infty$).

Для обеспечения существования классического решения проведена проверка согласованности заданных непрерывных граничных условий Дирихле (3) в угловых точках прямоугольной области $\bar{G}$:

1. Точка $(0, 0)$: $u(0, y)|_{y=0} = 0^2 = 0$ и $u(x, 0) = 0$.
2. Точка $(\pi, 0)$: $u(\pi, y)|_{y=0} = -0^2 = 0$ и $u(x, 0) = 0$.
3. Точка $(0, 1)$: $u(0, y)|_{y=1} = 1^2 = 1$ и $u(x, 1)|_{x=0} = \cos(0) = 1$.
4. Точка $(\pi, 1)$: $u(\pi, y)|_{y=1} = -1^2 = -1$ и $u(x, 1)|_{x=\pi} = \cos(\pi) = -1$.

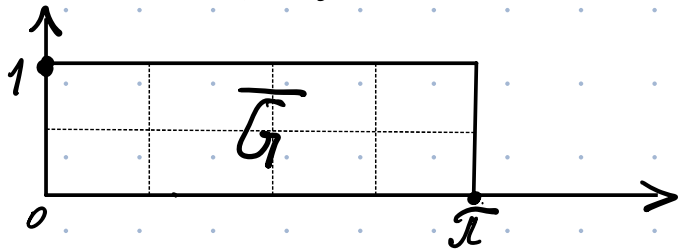
Таким образом, граничные условия непрерывно переходят друг в друга на стыках границ. Следовательно, поставленная краевая задача имеет единственное регулярное решение.

Метод Зейделя применим для решения систем линейных алгебраических уравнений, к которым путём дискретизации сводятся эллиптические дифференциальные уравнения второго порядка. Для задачи Пуассона, аппроксимированной на равномерной сетке, матрица системы обладает диагональным преобладанием и является симметричной и положительно определённой.

4. Тестовый пример с детальными расчётами

Решаем задачу (1) на области (2) с граничным условием (3).

$$\exists N_x = 4, N_y = 2$$



Шаги сетки:
 $h_x = \frac{\pi}{4}, h_y = \frac{1}{2}$

Узлы сетки:

$$x_i = \frac{\pi i}{4}, i = \overline{0, 4}$$

12 внеш. узлов, 3 внутр.

$$y_i = \frac{i}{2}, i = \overline{0, 2}$$

Граничные значения находим из усл. (3):

$$1) v_{0,0} = u(0,0) = 0$$

$$2) v_{0,1} = u(0,0,5) = 0,25$$

$$3) v_{0,2} = 1^2 = \cos 0 = 1$$

$$4) v_{1,2} = \cos \frac{\pi}{4} = \frac{\sqrt{2}}{2} \approx 0,707$$

$$5) v_{2,2} = \cos \frac{\pi}{2} = 0$$

$$6) v_{3,2} = \cos \frac{3\pi}{4} \approx -0,707$$

$$7) v_{4,2} = -1$$

$$8) v_{4,1} = -0,25$$

$$9-12) v_{4,0} = 0 = v_{3,0} = v_{2,0} = v_{1,0}$$

Начальное приближение $v^{(0)} = 0$

Итерация 1

1) узел $v_{1,1}$

По формуле (6):

$$v_{1,1}^{(1)} \approx \frac{\frac{0 + 0,25}{(\frac{\pi}{4})^2} + \frac{0 + 0,707}{(\frac{1}{2})^2} - (2 - \frac{1}{4}) \cos(\frac{\pi}{4})}{\frac{2}{(\frac{\pi}{4})^2} + \frac{2}{(\frac{1}{2})^2}} = 0,178$$

2) ужен $v_{2,1}$

$$v_{2,1}^{(1)} \approx \frac{\frac{0+0,178}{(\frac{\pi}{4})^2} + \frac{0+0}{(\frac{1}{2})^2}}{\frac{2}{(\frac{\pi}{4})^2} + \frac{2}{(\frac{1}{2})^2}} = 0,0257$$

3) ужен $v_{3,1}$

$$v_{3,1}^{(1)} \approx \frac{\frac{-0,25+0,0257}{(\frac{\pi}{4})^2} + \frac{0-0,707}{\frac{1}{4}} - (2-\frac{1}{4})\cos(\frac{3\pi}{4})}{11,242} = -0,174$$

$$v_{1,1}^{(1)} \approx 0,178; v_{2,1}^{(1)} \approx 0,0257; v_{3,1}^{(1)} \approx -0,174$$

Итерация 2

1) ужен $v_{1,1}$

$$v_{1,1}^{(2)} \approx \frac{\frac{0,0257+0,25}{0,617} + \frac{0+0,707}{\frac{1}{4}} - 1,75\cos(\frac{\pi}{4})}{11,242} = 0,1813$$

2) ужен $v_{2,1}$

$$v_{2,1}^{(2)} \approx \frac{\frac{-0,174+0,1813}{0,617}}{11,242} = 0,0011$$

3) ужен $v_{3,1}$

$$v_{3,1}^{(2)} \approx \frac{\frac{-0,25+0,0011}{0,617} + \frac{0-0,707}{\frac{1}{4}} - 1,75\cos(\frac{3\pi}{4})}{11,242} = -0,1774$$

$$v_{1,1}^{(2)} \approx 0,1813; v_{2,1}^{(2)} \approx 0,0011; v_{3,2}^{(2)} \approx -0,1774$$

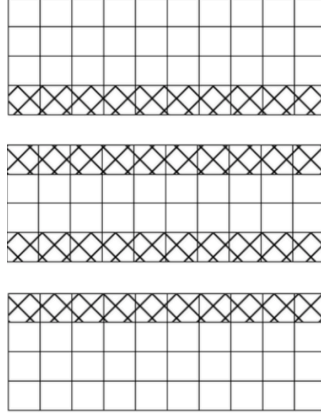


Рис. 2.1. Одномерная столбцовая декомпозиция

5. Подготовка контрольных тестов для иллюстрации метода

Во всех экспериментах значение точности было принято за 10^{-6} . В ходе выполнения работы проводилось две группы вычислительных экспериментов:

1. Для исследования зависимости времени выполнения от размера сетки задан дискретный набор из двух групп по 20 расчетных сеток:
 - Фиксированные значения $N_x = 40$ и $N_x = 100$ при изменении N_y от 10 до 200 с целью изоляции влияния дискретизации по оси Y .
 - Фиксированные значения $N_y = 40$ и $N_y = 100$ при изменении N_x от 10 до 200 с целью определения чувствительности алгоритма к декомпозиции полос по оси X .

В данном эксперименте число параллельных процессов было одним и тем же для всех сеток и равным 2.

2. Исследование зависимости времени, ускорения и эффективности от числа параллельных процессов. Здесь была зафиксирована сетка 80×400 узлов.

Ускорение параллельного алгоритма S_p находилось по формуле

$$S_p = \frac{T_1}{T_p},$$

где T_1 — время выполнения программы на одном процессе, а T_p — время выполнения задачи на p параллельных процессах.

Эффективность параллелизации E_p , характеризующая долю полезного использования вычислительных ресурсов, определялась следующим образом:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p},$$

где p — число запущенных параллельных процессов MPI ($p = \overline{1, 12}$).

Ограничение максимального количества параллельных процессов значением $p = 12$ обусловлено количеством доступных ядер процессора.

6. Модульная структура программы

Дерево проекта приведено на [Листинге 1](#).

Перейдём к подробному описанию файлов и функций программного комплекса:

- `tools.c`

- `double **alloc_matrix(int m, int n)` — выделение динамической памяти для матрицы размера $m \times n$;
- `void free_matrix(double **A, int n)` — освобождение памяти, занятой матрицей из n строк;
- `double max_in_matrix_diff(double** A, double** B, int m, int n)` — вычисление максимального модуля разности элементов двух матриц;
- `void init_matrix(double** A, int m, int n, double value)` — инициализация всех элементов матрицы заданным значением;
- `void swap_matrices(double ***A, double ***B)` — быстрый обмен указателей двух матриц текущей и предыдущей итераций.

- `seidel.c`

- `double **seidel(double border_x, double border_y, int Nx, int Ny, double eps, ...)` — функция блочно-параллельного метода Зейделя с обменом фиктивными слоями через `MPI_Sendrecv` и глобальным сбором матрицы решения через `MPI_Gatherv`.

- `functions.c`

- `double f(double x, double y)` — вычисление значения функции правой части уравнения Пуассона;
- `double u_0y(double y), double u_piy(double y)` — задание аналитических граничных условий на левой и правой границах;
- `double u_x0(double x), double u_x1(double x)` — задание аналитических граничных условий на нижней и верхней границах.

- `study.c`

- `void study_time_vs_grid_size(int rank, int size)` — изучение зависимости времени выполнения метода от размеров расчетной сетки ;

```
└─ src/
    ├── CMakeLists.txt
    ├── main.c
    ├── def.h
    ├── seidel.c
    ├── seidel.h
    ├── study.
    ├── study.h
    ├── test.
    ├── test.h
    ├── functions.
    ├── functions.h
    ├── tools.c
    ├── tools.h
    ├── build/
    │   └─ seidel
    └─ results/
        ├── scaling_data.csv
        ├── time_vs_grid.csv
        └─ solution.csv
```

Листинг 1: Структура программного комплекса

- `void study_scaling_performance(int rank, int size)` — расчет времени, ускорения и эффективности параллелизации на фиксированной сетке 80×400 при изменении числа процессов от 1 до 12 с записью полученных значений.

- `test.c`

- `void run_test_case(int rank)` — однократный запуск алгоритма на сетке тестового примера [раздела 4](#).

- `main.c`

- `int main(int argc, char **argv)` — инициализация параллельной среды MPI, запуск расчетных модулей и корректное освобождение ресурсов.

7. Численный анализ решения задачи

7.1. Графики решения задачи

Приведём на [Рис. 7.1](#) двумерный и трёхмерный графики численного решения задачи.

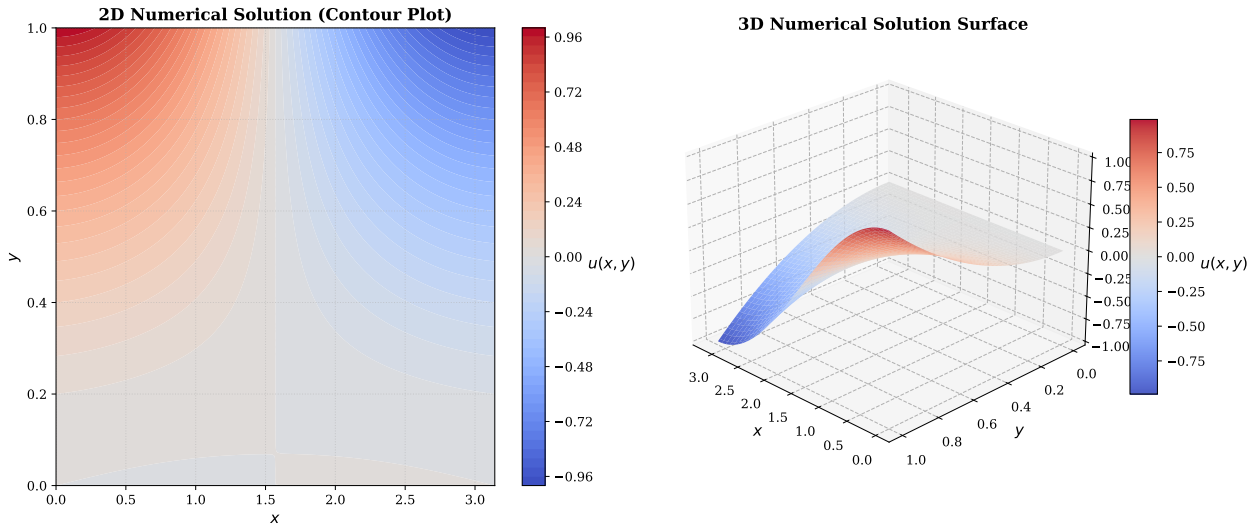


Рис. 7.1. Двумерный и трёхмерный графики численного решения

7.2. Исследование параллельной реализации

На Рис. 7.2 можно видеть график зависимости времени исполнения программы от высоты изображения N_y при его фиксированной ширине N_x .

Теперь зафиксируем ширину изображения N_y и построим график зависимости времени исполнения от его высоты N_x . Этот график приведён на Рис. 7.3.

В характере кривых времени исполнения видны явные различия, обусловленные в первую очередь выбором одномерной столбцовой декомпозиции расчётной области вдоль оси X . При фиксированном N_x геометрия разделения области на параллельные полосы остаётся неизменной, что стабилизирует число межпроцессных коммуникаций. В результате наблюдается гладкая параболическая зависимость времени от N_y , вызванная исключительно линейным ростом объёма локальных вычислений. Напротив, при фиксированном N_y увеличение N_x непосредственно изменяет число столбцов внутри локальной подобласти каждого процесса.

Перейдём к фиксированной сетке и анализу временных характеристик от числа параллельных процессов. График зависимости времени исполнения программы представлен на Рис. 7.4. Полученная кривая достигает минимального значения $T \approx 10$ сек при $p = 9$, после чего выигрыша во времени исполнении уже не наблюдается.

Построим на Рис. 7.5 график ускорения от числа параллельных процессов. Полученная кривая имеет максимум $S_p \approx 3.8$ при $p = 9$. На отрезке $p = \overline{1,5}$ наблюдается устойчивый рост, что обусловлено значительным преобладанием вычислительной нагрузки над накладными расходами на коммуникацию. Однако при $p > 5$ темп роста существенно замедляется, а при $p > 9$ время ускорение начинает снижаться, так как объём полезных арифметических вычислений на итерации становится малым по сравнению с коммуникационными затратами.

На Рис. 7.6 представлен график зависимости эффективности параллелизации E_p от числа параллельных процессов. Экспериментальная кривая демонстрирует монотонное

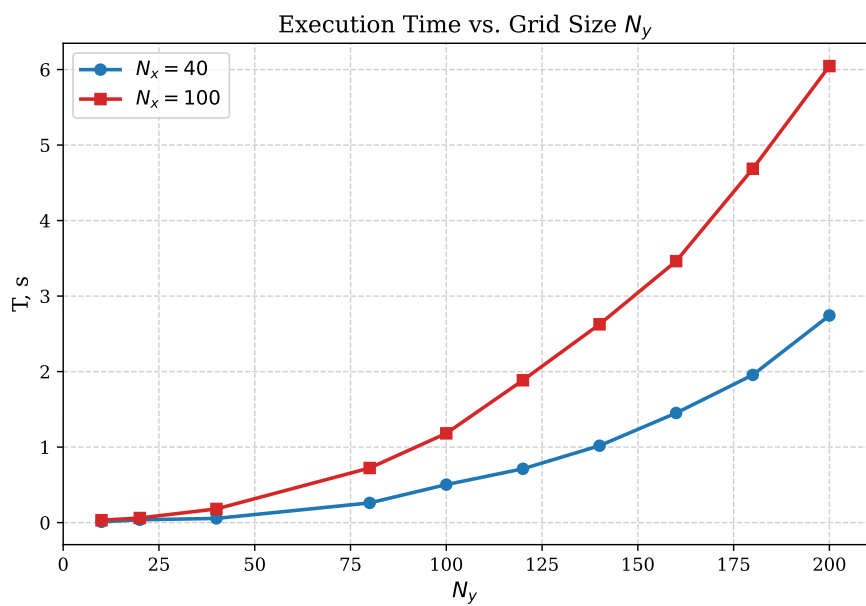


Рис. 7.2. Зависимость времени исполнения программы от высоты изображения

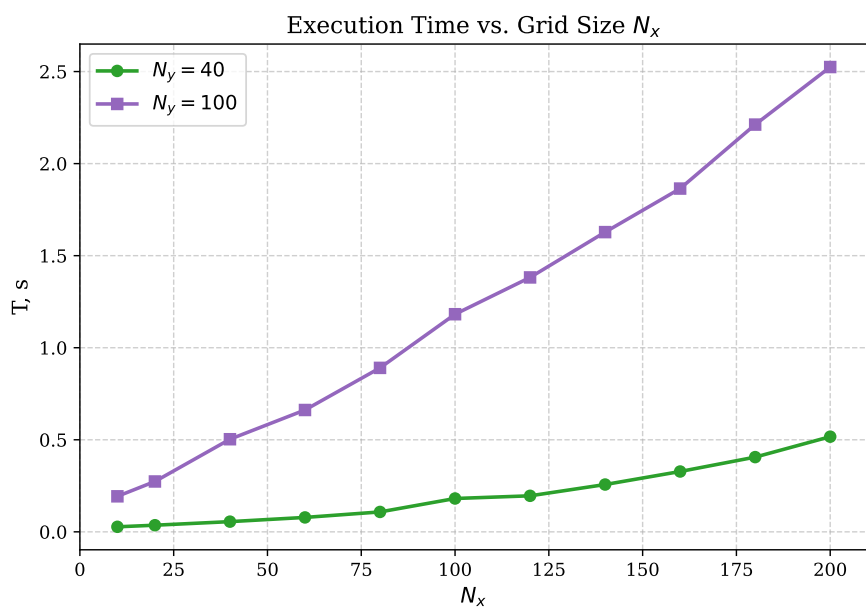


Рис. 7.3. Зависимость времени исполнения программы от ширины изображения

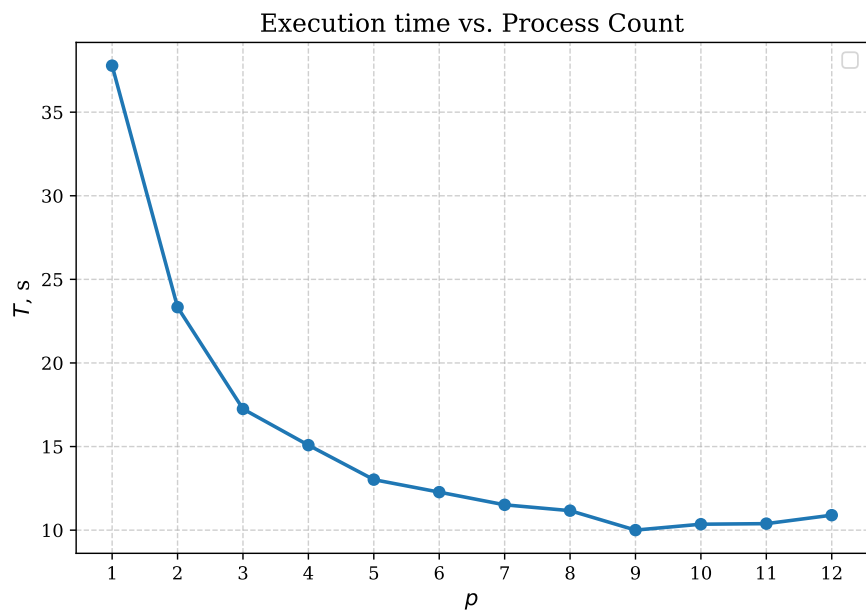


Рис. 7.4. Зависимость времени исполнения программы от числа параллельных процессов

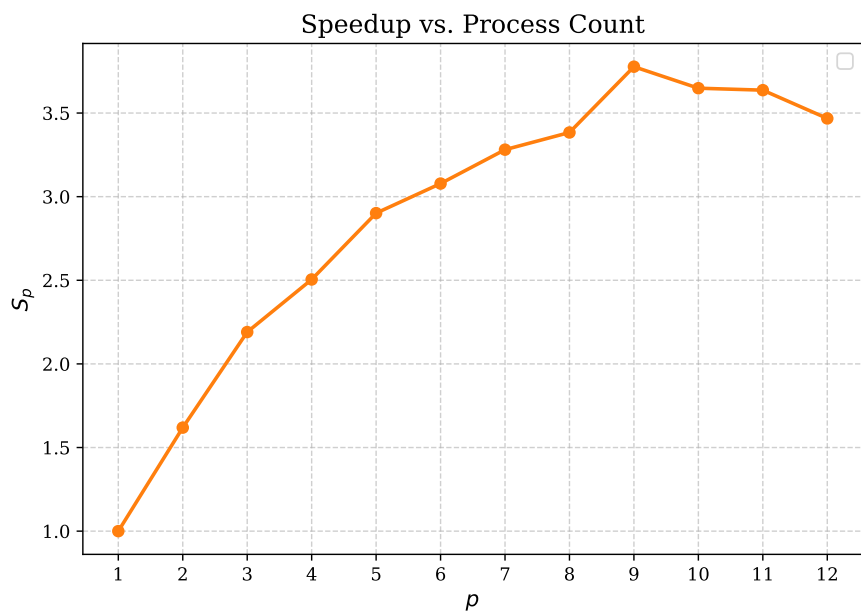


Рис. 7.5. Зависимость ускорения от числа параллельных процессов

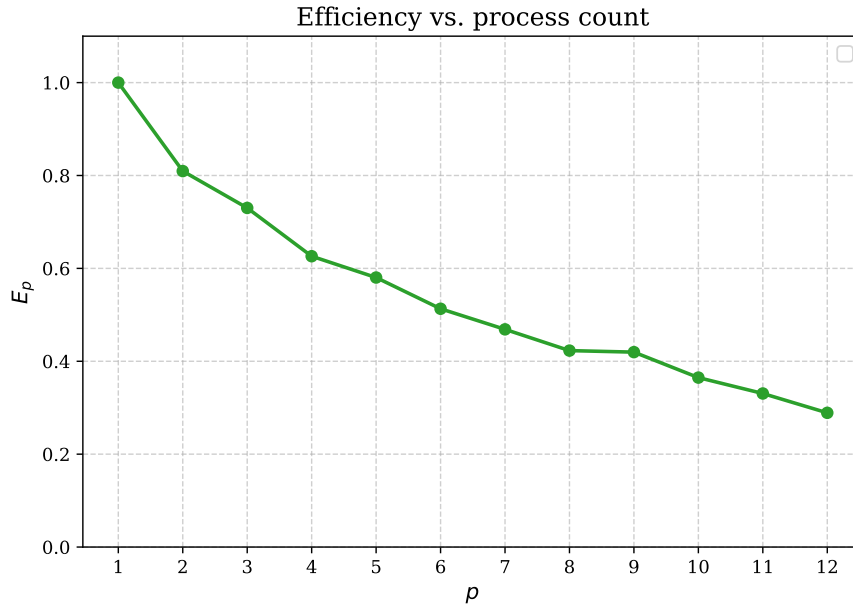


Рис. 7.6. Зависимость эффективности параллелизации от числа параллельных процессов

падение эффективности с максимального значения $E_1 = 1.0$ до критически низкого уровня $E_{12} \approx 0.27$.

8. Выводы

В ходе выполнения лабораторной работы было реализовано численное решение двумерного уравнения Пуассона с граничными условиями Дирихле параллельным методом Зейделя с использованием технологии MPI, а также проведено исследование масштабируемости и производительности алгоритма.

1. Предварительный анализ гладкости правой части задачи и проверка условий согласованности граничных функций в угловых точках расчетной области подтвердили существование и единственность классического решения. Построенные двумерный и трёхмерный графики численного решения продемонстрировали совпадение с точным аналитическим решением задачи.
2. Эксперимент на дискретном наборе из 40 расчетных сеток подтвердил определяющее влияние выбранного способа декомпозиции на характер времени выполнения программы.
3. Исследование показателей эффективности на фиксированной сетке 80×400 при изменении числа параллельных процессов p от 1 до 12 выявило выход на насыщение и пиковое ускорение $S_p \approx 3.8$ при $p = 9$. Установлено, что существенный рост скорости исполнения и высокая эффективность параллелизации ($E_p \geq 0.65$) достигаются в основном на малом числе процессов, когда объём локальной арифметики существенно доминирует над затратами на межпроцессное взаимодействие.

4. Выявлено, что с ростом числа параллельных процессов происходит спад эффективности параллелизации (вплоть до $E_{12} \approx 0.27$).

9. Примечания

Код реализации метода доступен на [Github](#).